RFC1
Tablas: solicitud, viaje

Atributos de join: solicitud.id_viaje = viaje.id_viaje

Explicación: Se usa INNER JOIN porque toda solicitud tiene un viaje (la FK es obligatoria) y se necesita la fecha de salida del viaje para filtrar el periodo. No se usa LEFT JOIN porque no existe una tabla de localidades de la cual conservar filas sin solicitudes.

Supuestos: la demanda cuenta las solicitudes de cualquier estado; el periodo se mide por la fecha de salida del viaje, porque solicitud no tiene fecha propia.

```
1  SELECT s.localidad,
2         COUNT(*) AS total_solicitudes
3  FROM   solicitud s
4  JOIN   viaje v ON v.id_viaje = s.id_viaje
5  WHERE  v.fecha_hora_estimada_salida >= DATE '2026-08-01'
6  AND    v.fecha_hora_estimada_salida <  DATE '2026-12-31' + 1
7  GROUP BY s.localidad
8  ORDER BY total_solicitudes DESC, s.localidad;
```

LOCALIDAD	TOTAL_SOLICITUDES
Usaquén	42
Teusaquillo	37
Suba	35
Chapinero	32
Kennedy	30
Engativá	21
Puente Aranda	20
Rafael Uribe Uribe	20
Barrios Unidos	17
Fontibón	16
Ciudad Bolívar	14
Usme	10
Santa Fe	6

13 rows returned in 0.00 seconds Download

RFC2
Tablas: solicitud, viaje

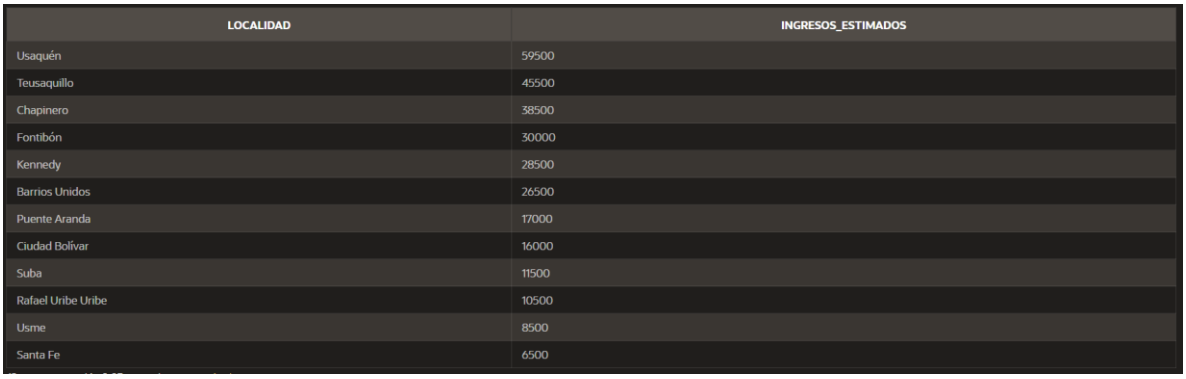
Atributos de join: solicitud.id_viaje = viaje.id_viaje

Explicación: Se usa INNER JOIN porque se necesita el costo del viaje de cada solicitud. Solo cuentan las solicitudes en estado aceptada, ya que son las que generan ingreso. El ingreso estimado de una localidad es la suma de costo_del_viaje de esas solicitudes (el precio es el mismo para todos los pasajeros del viaje).

Supuestos: el periodo se mide por la fecha de salida del viaje; la localidad es la de recogida registrada en la solicitud

Supuesto de RFC1 y RFC2: Al no existir una tabla de localidades, solo muestran localidades con al menos una solicitud.

```
SELECT s.localidad,
       SUM(v.costo_del_viaje) AS ingresos_estimados
FROM   solicitud s
JOIN   viaje v ON v.id_viaje = s.id_viaje
WHERE  s.solicitud_aceptada = 'aceptada'
AND    v.fecha_hora_estimada_salida >= DATE '2026-08-01'
AND    v.fecha_hora_estimada_salida <  DATE '2026-12-31' + 1
GROUP BY s.localidad
ORDER BY ingresos_estimados DESC, s.localidad;
```



LOCALIDAD	INGRESOS_ESTIMADOS
Usaquén	59500
Teusaquillo	45500
Chapinero	38500
Fontibón	30000
Kennedy	28500
Barrios Unidos	26500
Puente Aranda	17000
Ciudad Bolívar	16000
Suba	11500
Rafael Uribe Uribe	10500
Usme	8500
Santa Fe	6500

RFC3

Tablas: viaje, conductor (y solicitud solo cuando se filtra por localidad)

Atributos de join: viaje.doc_identidad_conductor = conductor.documento_de_identidad

Explicación: Se usa INNER JOIN porque el resultado son viajes realizados: un conductor sin viajes no aporta filas a una serie en el tiempo. Para el filtro por localidad se usa EXISTS sobre solicitud y no un JOIN, para no contar el viaje varias veces si tiene varios pasajeros en la misma localidad.

Supuestos: “realizados” son los viajes en estado completado; la localidad de un viaje se toma de sus solicitudes aceptadas, porque parada_ruta_viaje guarda barrio y no localidad.

```sql
SELECT c.documento_de_identidad,
       c.nombre || ' ' || c.apellidos            AS conductor,
       TRUNC(v.fecha_hora_estimada_salida, 'MM')  AS mes,
       COUNT(*)                                  AS viajes
FROM   viaje v
JOIN   conductor c ON c.documento_de_identidad = v.doc_identidad_conductor
WHERE  v.estado_del_viaje = 'completado'
AND    v.fecha_hora_estimada_salida >= DATE '2026-08-01'
AND    v.fecha_hora_estimada_salida <  DATE '2026-12-31' + 1
GROUP BY c.documento_de_identidad, c.nombre, c.apellidos,
         TRUNC(v.fecha_hora_estimada_salida, 'MM')
ORDER BY c.documento_de_identidad, mes;
```

DOCUMENTO_DE_IDENTIDAD	CONDUCTOR	MES	VIAJES
2	Camila Rodríguez Castro	10/1/2026	1
4	Valentina López Rincón	10/1/2026	1
5	Andrés García Rodríguez	10/1/2026	1
8	Laura Torres Herrera	10/1/2026	1
9	Mateo Díaz Quintero	10/1/2026	1
10	Sofía Vargas Ramírez	9/1/2026	1
11	Sebastián Castro Jiménez	9/1/2026	1
13	Nicolás Rojas Peña	10/1/2026	1
16	Natalia Gutiérrez Cárdenas	9/1/2026	1
18	Carolina Sánchez Martínez	10/1/2026	1